AutoChecker ： 从规范与漏洞修复中自动生成代码检查器

计理强[1], 黄进豪[1], 刘珺[1], 燕季薇[1], 张健[1]

[1]中国科学院软件研究所，北京

摘要

针对现有静态分析工具难以覆盖多样化检测需求、人工开发静态代码检查器成本较高的问题，本文设计并实现了智能静态代码检查器生成系统 **AutoChecker** 。系统支持两条生成路线：规范与测试用例驱动的检查器生成，以及漏洞与补丁驱动的检查器生成，能够从自然语言规范、代码示例及历史漏洞信息中自动生成可执行的静态代码检查器。为提高生成质量，系统结合检测逻辑引导的知识检索与 Agentic RAG，利用专业知识辅助检查器生成；通过补丁引导的漏洞机制分析提取漏洞机制，并结合真实工具链验证与差分谓词探测，实现候选检查器的迭代修复。在实际开源项目的应用中，**AutoChecker** 生成的检查器在 13 个开源代码仓库中发现了 16 个代码漏洞，均获得项目开发者确认，其中两项漏洞获分配 CVE 编号（CVE-2026-103754 和 CVE-2026-71364）。上述结果展示了 **AutoChecker** 在自动化检查器开发和实际软件缺陷发现方面的应用价值。

1 项目背景和创新点

随着软件系统在信息基础设施、工业控制和公共服务等关键领域的广泛应用，其规模与复杂度持续增长，软件缺陷引发的可靠性与安全问题日益突出。静态分析技术能够在不执行程序的情况下检测潜在代码缺陷，已被广泛应用于代码审查、持续集成和软件安全审计等场景。以 CodeQL、PMD 和 Clang-Tidy 为代表的静态分析工具提供了丰富的预定义检查规则，能够识别常见的编码违规、程序缺陷和安全漏洞。然而，预定义规则难以全面覆盖实际开发过程中多样化且持续演化的软件质量保障需求。一方面，不同组织和项目具有特定的编码规范、框架约束及业务安全要求，需要针对具体开发场景实施定制化检查；另一方面，新披露的安全漏洞及其修复补丁不断揭示新的缺陷模式，对静态分析工具的规则覆盖范围和更新能力提出了更高要求。因此，如何将多样化的软件质量保障需求及时转化为可执行的静态代码检查器，成为提升静态分析技术适用性与有效性的重要问题。本文将不同静态分析工具中用于实现特定检测逻辑的规则或查询统一称为静态代码检查器，以下简称“检查器”。

在实际软件开发过程中，定制化检查需求通常以自然语言规范、代码示例或历史缺陷记录等形式存在。国家与行业标准、企业编码规范及开发手册通常通过自然语言描述代码应满足的约束条件，并辅以合规与违规示例说明具体的检查要求。此外，公开披露的安全漏洞、CVE 条目及其修复补丁也为识别新的缺陷模式提供了重要依据。这些历史缺陷记录反映了特定程序上下文中的错误实现及相应修复方式，可用于分析缺陷产生的原因并归纳潜在的违规模式。然而，这些信息通常缺乏静态分析工具所需的形式化检测逻辑，无法直接作为可执行的检查器。因此，需要从不同形式的需求描述和缺陷实例中提取可检测的程序特征，并将其转化为目标静态分析工具支持的检查器实现。

传统静态代码检查器开发主要依赖具有程序分析经验的专业人员，需要人工完成检测逻辑设计、工具接口适配及测试验证，开发成本较高，难以快速响应持续增长的定制化检查需求。大语言模型为检查器自动生成提供了新的技术途径，但将自然语言规范或历史漏洞转化为可执行且具有准确检测语义的检查器仍面临多方面挑战。首先，静态分析框架的专业知识和接口约束增加了检查器生成的难度。由于不同工具在抽象语法树表示、控制流与数据流建模等方面存在差异，大语言模型由于缺乏充分的框架知识支持，容易生成包含错误 API 调用、类型不匹配或不合法分析逻辑的代码，导致检查器无法编译或执行。其次，检查器的有效性取决于对检测需求的准确理解。自然语言规范可能存在条件描述不完整或语义歧义等问题，而漏洞报告与修复补丁通常仅反映特定程序上下文中的缺陷表现。若不能准确识别检测条件、分析漏洞产生的根本原因，生成的检查器便可能依赖局部代码特征，难以识别其他实现形式下的同类缺陷。最后，有限的验证反馈难以直接指导检查器的有效修复。候选检查器可能存在编译错误、漏报或误报，这些问题可能源于 API 使用错误、程序语义建模偏差或检测条件设置不当，仅依靠原始错误信息或检测结果进行反复生成，难以准确定位问题并确定有效的修复方向。因此，检查器自动生成不仅需要解决代码合成问题，还需要解决专业知识获取、检测语义理解及反馈驱动的错误诊断与修复等问题。

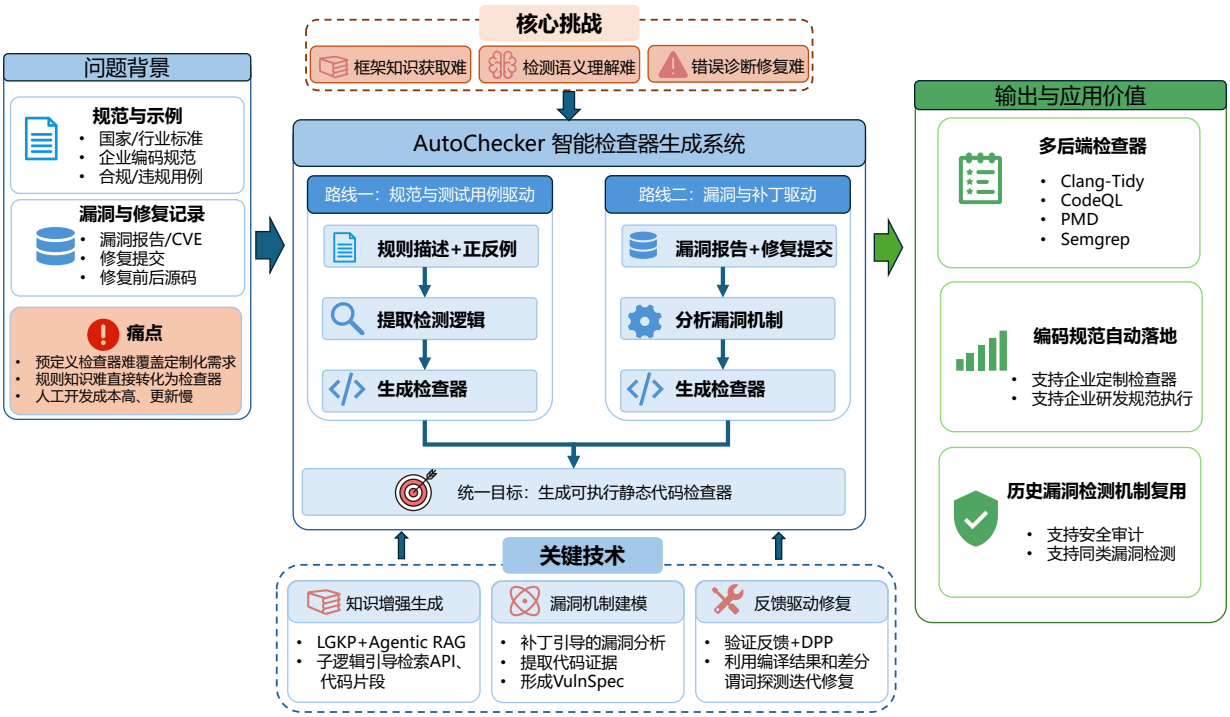


图 1: **AutoChecker** 的问题背景、核心挑战与技术创新

针对上述挑战，本项目设计并实现了智能静态代码检查器生成系统 **AutoChecker**，将自然语言规范、代码测试用例及历史漏洞信息自动转化为可执行的静态代码检查器。系统支持两条互补的生成路线：一是**规范与测试用例驱动的检查器生成**，根据自然语言规则描述及合规、违规测试用例提取检测逻辑，面向 Clang-Tidy、CodeQL、PMD 和 Semgrep 等静态分析工具生成检查器；二是**漏洞与补丁驱动的检查器生成**，从公开漏洞与修复补丁出发，分析漏洞产生的根本原因，生成能够识别同类缺陷的 CodeQL 查询。两条路线分别面向编码规范的自动化落地和历史漏洞检测能力的复用，共同构成从软件质量与安全知识到可执行检查器的自动化生成流程。

为实现上述目标，**AutoChecker** 围绕检查器生成过程中的知识获取、检测语义理解和错误诊断修复设计了三项关键技术。在**知识获取**方面，提出检测逻辑引导的知识检索（Logic-Guided Knowledge Retrieval，LGKR）与自主检索增强生成（Agentic RAG）相结合的方法。LGKR 将检测需求分解为多个具有明确语义的检测子逻辑，分别检索与各子逻辑相关的静态分析 API、基础分析操作及实现示例，为检查器生成提供针对性的框架知识；在生成和修复过程中，智能体还可根据当前任务中尚未解决的问题自主发起补充检索。在**检测语义理解**方面，系统针对两条生成路线采用不同的分析策略：对于规范与测试用例驱动的检查器生成路线，结合规则描述与合规、违规示例提取检测条件及其适用边界；对于漏洞与补丁驱动的检查器生成路线，则采用补丁引导的漏洞机制分析方法，由分析智能体从修复差异出发，探索漏洞版本与修复版本的相关源码，收集能够支持漏洞成因分析的代码证据，并提取漏洞触发条件、根本原因及修复机制，形成结构化漏洞规格（Vulnerability Specification，简称 VulnSpec），为后续检查器生成提供语义约束，降低对特定补丁或局部代码特征的依赖。在**错误诊断与修复**方面，系统构建了验证反馈驱动的迭代修复机制，将候选检查器置于真实静态分析工具链中执行验证。对于编译或语法错误，系统结合诊断信息，通过 Agentic RAG 检索相关 API 文档和使用示例，修复接口使用及实现问题；对于检测结果与预期不符的情况，则依据相应测试反馈调整检测逻辑。其中，针对 CodeQL 安全检查器，进一步提出差分谓词探测（Differential Predicate Probing，DPP）与反馈驱动的迭代修复机制：在漏洞版本与修复版本上执行完整查询后，利用 CodeQL 的局部表达式求值能力，对关键谓词和中间关系进行双版本探测，通过分析匹配数量、代表性结果及版本差异，为检测语义错误的定位提供细粒度反馈，指导生成智能体对候选检查器进行针对性修复。

综上，**AutoChecker** 的主要创新体现在以下三个方面：

1. **规范与漏洞双路线的检查器自动生成：**设计了面向不同检测需求的统一检查器生成框架，支持从自然语言规范与测试用例、历史漏洞与修复补丁两类输入自动生成静态代码检查器，分别实现编码规范的自动化检查与历史漏洞检测能力的复用。

2. **规范与测试用例驱动的知识增强生成方法：**针对静态分析框架知识复杂、通用检索难以准确覆盖具体检测条件的问题，提出检测逻辑引导的知识检索（LGKR）与 Agentic RAG 相结合的方法。通过将检测需求分解为多个子逻辑，分别检索相关 API、分析操作及实现示例，并支持 Agent 按需补充领域知识，为候选检查器生成提供针对性的知识支持。

3. **漏洞与补丁驱动的漏洞分析与差分反馈修复方法：**针对漏洞机制难以从补丁中直接提取、检查器语义错误难以定位的问题，设计补丁引导的漏洞机制分析方法，通过 Agent 源码探索与证据提取构建 VulnSpec；进一步提出差分谓词探测（DPP）机制，结合完整检查器检测结果与关键谓词的双版本匹配反馈，指导 Agent 对候选检查器进行针对性修复。

上述创新从多来源检查需求的统一处理出发，分别通过知识增强生成和漏洞机制分析与差分反馈修复，支撑两条检查器生成路线的自动化实现。

2 应用场景

AutoChecker 面向软件开发过程中多样化、持续演化的代码质量与安全检查需求，主要应用于编码规范自动化实施和历史漏洞检测能力复用两类场景，同时支持多种静态分析工具的规则实现与研发流程集成。

1. **编码规范自动化实施**针对国家及行业标准、企业编码规范和项目特定的开发约束，**AutoChecker** 利用自然语言规则描述及合规、违规代码示例，自动生成相应的静态代码检查器。该方式既可用于常见编码规范的自动化检查，也可面向企业自研框架、内部 API 和特定业务逻辑构建定制化检查器，将原本依赖人工审查的规范要求转化为可自动执行的检测能力。

2. **漏洞检测与安全规则开发**针对公开披露的安全漏洞及其修复补丁，**AutoChecker** 通过分析漏洞版本与修复版本的代码差异，提取漏洞产生的根本原因及关键检测条件，生成面向 CodeQL 的安全检查器。该场景旨在将单次漏洞修复中蕴含的缺陷模式转化为可复用的检测能力，用于识别代码库中可能存在的同类漏洞，辅助安全审计与漏洞变体检测。

3. **多工具适配与研发流程集成 AutoChecker** 支持面向 Clang-Tidy、CodeQL、PMD 和 Semgrep 等静态分析工具生成检查器，可根据项目使用的分析工具及编程语言选择相应的生成后端。生成并验证通过的检查器可进一步集成至持续集成、代码审查和安全扫描流程，在代码提交及版本发布前自动识别潜在的质量与安全问题，降低定制化检查能力的开发和维护成本。

3 技术方案

AutoChecker 的技术方案如图 2 所示，主要包括规范与测试用例驱动的检查器生成、漏洞与补丁驱动的检查器生成两条路线。两条路线采用不同的检测需求分析方法：前者结合自然语言规则描述与合规、违规测试用例提取检测逻辑，通过检测逻辑引导的知识检索（LGKR）与 Agentic RAG 获取目标静态分析框架的相关知识，生成候选检查器；后者以公开漏洞、修复补丁及修复前后的源码为输入，通过分析智能体自主探索代码仓库、分析漏洞根因并提取源码证据，形成结构化漏洞规格 VulnSpec，再据此生成面向 CodeQL 的候选检查器。

两条路线均采用验证反馈驱动的迭代生成机制，但验证方式有所不同。规范与测试用例驱动的检查器生成路线通过编译验证及测试套件执行，依据编译错误、漏报和误报反馈修复候选检查器；漏洞与补丁驱动的检查器生成路线则在漏洞版本与修复版本上执行检查器，验证其能否识别目标缺陷并区分修复前后的程序行为。当检测结果不符合预期时，系统进一步利用差分谓词探测（DPP）获取关键检测逻辑的双

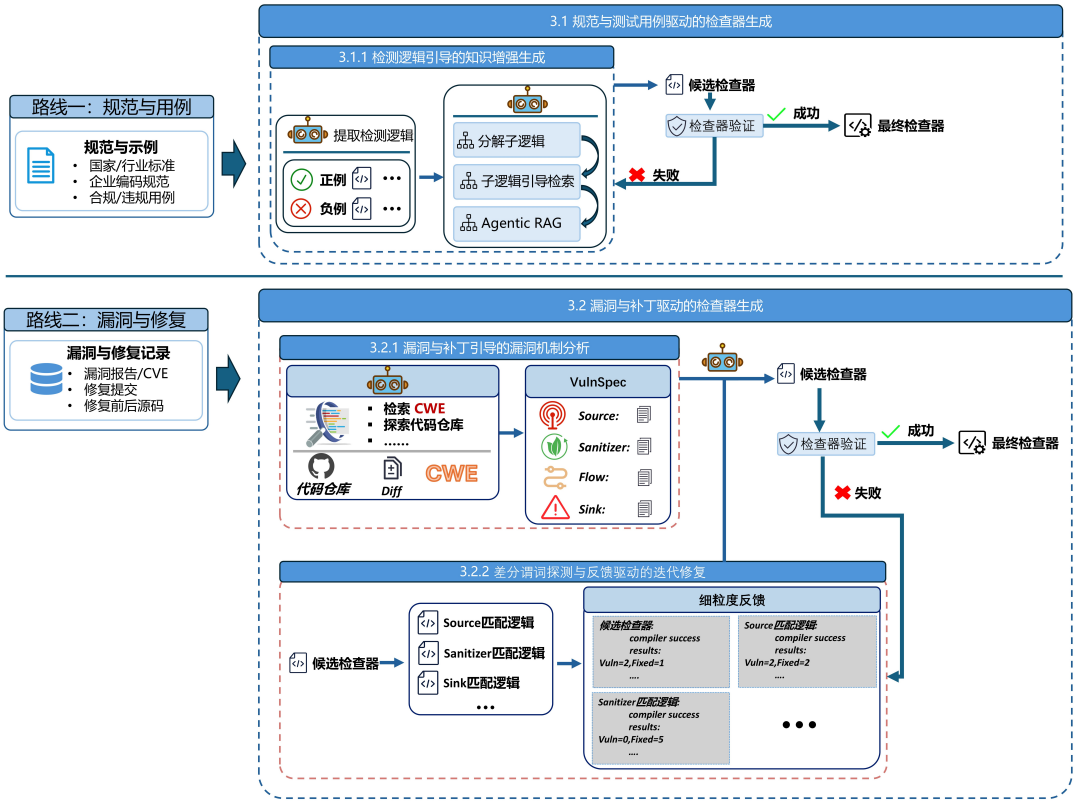


图 2: **AutoChecker** 技术方案总体框架

版本匹配结果，为生成智能体提供细粒度的语义诊断信息，指导候选检查器的针对性修复。通过上述流程，**AutoChecker** 将不同来源的检测需求转化为可在目标静态分析工具中执行、并经过相应验证的检查器。

3.1 规范与测试用例驱动的检查器生成

该路线以自然语言规则描述及合规、违规测试用例作为输入，旨在自动生成满足指定检测需求的静态代码检查器。其中，规则描述用于明确待检测的代码缺陷及相关约束，测试用例则提供具体的代码实例与预期检测结果，作为检查器生成和验证的依据。系统首先由大语言模型分析规则描述及测试用例，提取关键检测条件，并结合检测逻辑引导的知识增强生成方法，为生成智能体提供目标静态分析框架所需的 API、分析操作和实现示例。在此基础上，生成智能体生成候选检查器，并调用相应的静态分析工具进行编译和测试。知识增强生成方法的具体设计将在第 3.1.1 节介绍。

对于生成的候选检查器，系统根据验证结果引导生成智能体进行迭代修复。若检查器存在语法或编译错误，系统将编译诊断信息反馈给生成智能体，并结合知识增强检索获取相关 API 文档及实现示例，辅助修复接口调用、类型约束和代码实现等问题。若检查器能够成功编译，则在测试套件上执行检测，并将实际检测结果与预期结果进行比较，识别漏报和误报。对于漏报用例，系统向生成智能体提供未被正确识别的违规代码及其预期告警信息，引导其检查检测条件是否缺失或过于严格；对于误报用例，则提供被错误报告的合规代码及对应告警位置，引导其分析检测条件是否过于宽泛或缺少必要的约束。生成智能体根据这些反馈修改上一轮候选检查器，并重新执行完整测试套件，检查修复效果及是否引入新的误报或漏报。上述过程持续迭代，直至候选检查器通过指定测试套件的验证，或达到预设的迭代上限。最终，系统输出生成的检查器及其验证结果。

3.1.1 检测逻辑引导的知识增强生成

静态代码检查器的生成需要准确使用目标分析框架提供的 API、程序分析操作及其组合方式。由于大语言模型对静态分析框架的专业知识掌握有限，生成的检查器容易出现错误 API 调用、类型不匹配或分析逻辑不正确等问题。传统检索增强生成方法通常直接根据完整的规则描述检索相关知识，难以为规则中不同的检测条件提供针对性的实现参考。为此，**AutoChecker** 提出检测逻辑引导的知识检索（Logic-

Guided Knowledge Retrieval，LGKR）与自主检索增强生成（Agentic RAG）相结合的知识增强方法，为检查器生成提供所需的静态分析领域知识。

LGKR 首先利用大语言模型分析规则描述及合规、违规测试用例，将完整检测需求分解为若干具有明确语义的检测子逻辑，形成检测逻辑集合。每个子逻辑以自然语言描述一项具体的检测操作，例如定位程序实体、判断函数调用关系或检查控制流约束。系统以各子逻辑为检索单元，分别从静态分析知识库中获取相关 API、基础分析操作及实现示例，并将检索结果提供给生成智能体。通过将完整检测需求分解为具体的检测操作，LGKR 能够围绕不同检测条件分别组织检索，为候选检查器的实现提供针对性的知识参考。

在此基础上，生成智能体结合规则描述、测试用例及 LGKR 提供的领域知识生成候选检查器。考虑到预先检索的知识可能无法覆盖所有实现需求，系统进一步引入 Agentic RAG，使生成智能体能够根据当前生成任务及已有检索结果，自主判断是否需要补充知识，并构造检索请求获取相关 API 文档、分析操作或实现示例。LGKR 为检查器生成提供基于检测逻辑的初始知识，Agentic RAG 则根据实际生成需求动态补充知识，两者共同支持候选检查器的生成。

3.2 漏洞与补丁驱动的检查器生成

该路线以公开漏洞及其修复补丁为输入，旨在从具体漏洞实例中提取可复用的检测逻辑，并自动生成面向 CodeQL 的静态代码检查器。整个生成过程主要包括补丁引导的漏洞机制分析和检查器生成与迭代修复两个阶段。

在漏洞机制分析阶段，系统以修复补丁为起点，由分析智能体探索修复前后的代码仓库，结合补丁差异及相关源码上下文，分析漏洞的触发条件、根本原因和修复机制。为避免仅根据局部代码差异推断漏洞模式，分析智能体需要收集能够支持漏洞分析结论的源码证据，并据此生成结构化漏洞规格（Vulnerability Specification，VulnSpec）。VulnSpec 描述漏洞产生的关键程序行为及约束条件，为后续检查器生成提供明确的检测目标。补丁引导的漏洞机制分析方法将在第 3.2.1 节详细介绍。

在检查器生成阶段，系统根据 VulnSpec 所描述的漏洞机制，由生成智能体合成候选 CodeQL 检查器。生成过程中，系统利用检测逻辑引导的知识增强生成方法，为生成智能体提供相关的 CodeQL API、分析操作及实现示例，并允许其根据需要自主补充检索。候选检查器首先接受编译验证，编译通过后分别在漏洞版本与修复版本的 CodeQL 数据库上执行，检查其能否在漏洞版本中识别目标缺陷，同时避免在修复版本中继续报告相同漏洞。对于未通过验证的候选检查器，系统根据编译诊断或检测结果引导生成智能体进行迭代修复。其中，针对漏报和误报问题，系统进一步利用差分谓词探测获取关键谓词及中间关系的双版本执行结果，为定位检测逻辑错误提供细粒度反馈。相关知识增强方法与迭代修复机制分别在第 3.1.1 节和第 3.2.2 节介绍。最终，系统输出生成的检查器及其验证结果。

3.2.1 补丁引导的漏洞机制分析

漏洞修复补丁记录了程序从缺陷状态到修复状态的代码变化，但这些变化并不直接等价于漏洞产生的根本原因。一次提交可能同时包含安全修复、代码重构及其他无关修改，而漏洞的触发条件也可能涉及变更位置之外的程序上下文。为此，**AutoChecker** 设计了补丁引导的漏洞机制分析方法，由分析智能体结合修复提交、补丁差异及修复前后的代码仓库，逐步定位安全相关代码并分析漏洞产生的原因，最终生成结构化漏洞规格（Vulnerability Specification，VulnSpec）。

首先，分析智能体读取提交说明及代码差异（Diff），识别可能与漏洞修复相关的代码变更，排除格式调整、无关重构及其他非安全修改，确定需要进一步分析的函数或代码区域。随后，分析智能体阅读漏洞版本中相关函数的完整实现，并与修复版本中的对应代码进行比较，分析补丁修改了哪些程序行为，以及这些变化为何能够消除原有缺陷。对于仅凭变更位置无法解释的程序行为，分析智能体进一步检索相关函数调用、变量定义、数据依赖或配置约束，按需扩展源码分析范围，而不局限于补丁直接修改的代码。

在源码探索过程中，分析智能体重点分析漏洞的触发条件、存在缺陷的程序行为及其安全影响，并通过修复前后的实现差异验证对漏洞根因的判断。例如，对于缺失安全检查的漏洞，需要明确相关操作在何种条件下执行、原实现缺少何种必要约束，以及修复代码如何补充该约束；对于错误的数据处理逻

辑，则需要分析相关数据的来源、处理过程及其在缺陷位置的使用方式。分析过程中，系统记录支撑漏洞判断的源码证据，包括代码所属版本、文件路径及对应代码范围，使漏洞根因和修复机制能够与具体程序实现相互对应。当现有证据不足以解释漏洞产生的原因时，分析智能体继续探索相关代码，直至获得能够支持完整漏洞解释的必要证据，或达到预设的分析限制。

完成漏洞分析后，分析智能体根据已收集的源码证据，归纳漏洞名称、根本原因和修复机制，形成结构化漏洞规格 VulnSpec。该规格重点描述导致漏洞产生的程序行为及其必要条件，并保留相应的源码证据引用，为后续检查器生成提供明确的检测目标。与直接根据补丁中的代码增删生成检查器不同，VulnSpec 将具体修复实例中的漏洞机制与文件路径、函数名称等实现细节加以区分，使后续生成智能体能够依据漏洞语义构建检测逻辑，而非直接匹配特定提交中的代码特征。

3.2.2 差分谓词探测与反馈驱动的迭代修复

在检查器迭代生成过程中，编译错误可以通过诊断信息辅助定位，而检测语义错误通常难以仅凭最终检测结果进行分析。例如，当候选检查器未能识别漏洞版本中的目标缺陷时，可能是程序实体定位错误、数据流关系不完整或检测条件过于严格；当检查器在漏洞版本和修复版本中均报告目标缺陷时，则可能未能正确建模修复所涉及的安全约束。由于完整检查器的执行结果无法直接反映内部检测逻辑的匹配情况，Agent 难以判断应当修改哪些检测条件。为此，**AutoChecker** 提出差分谓词探测（Differential Predicate Probing，DPP）机制，利用 CodeQL 的局部表达式求值能力，为候选检查器的语义错误诊断提供细粒度反馈。

系统首先分别在漏洞版本与修复版本的 CodeQL 数据库上执行完整候选检查器，收集检测结果及对应的代码位置，并结合 VulnSpec 判断其是否满足预期检测行为。当完整检查器未能正确识别目标漏洞，或在修复版本中仍报告相应缺陷时，系统进一步对检查器内部的关键检测逻辑进行探测。具体而言，系统从候选检查器中选择与程序实体识别、数据流传播及安全约束等相关的谓词和局部表达式，利用 CodeQL 的局部表达式求值能力，分别获取这些中间逻辑在两个版本中的匹配数量及具体结果。对于能够映射到源码的匹配结果，系统进一步提取对应的代码位置与代码片段，以便分析各项检测条件实际匹配了哪些程序实体及其关系。

在错误修复阶段，系统将完整检查器的双版本检测结果与局部逻辑的探测结果共同提供给生成 Agent。完整检测结果用于说明候选检查器是否识别目标缺陷，以及是否在修复版本中产生误报；局部探测结果则提供关键谓词的匹配数量、匹配到的程序实体及对应源码片段，帮助 Agent 分析检测逻辑中的具体偏差。例如，当目标漏洞未被报告时，Agent 可以根据中间逻辑的匹配情况，判断问题可能出现在目标实体识别、数据流连接或过滤条件等环节；当修复版本仍被报告时，则可以结合双版本代码差异及谓词匹配结果，检查是否遗漏了必要的安全约束。Agent 在 VulnSpec 和源码证据的约束下，根据上述反馈修改存在问题的检测逻辑，而非重新生成整个检查器。

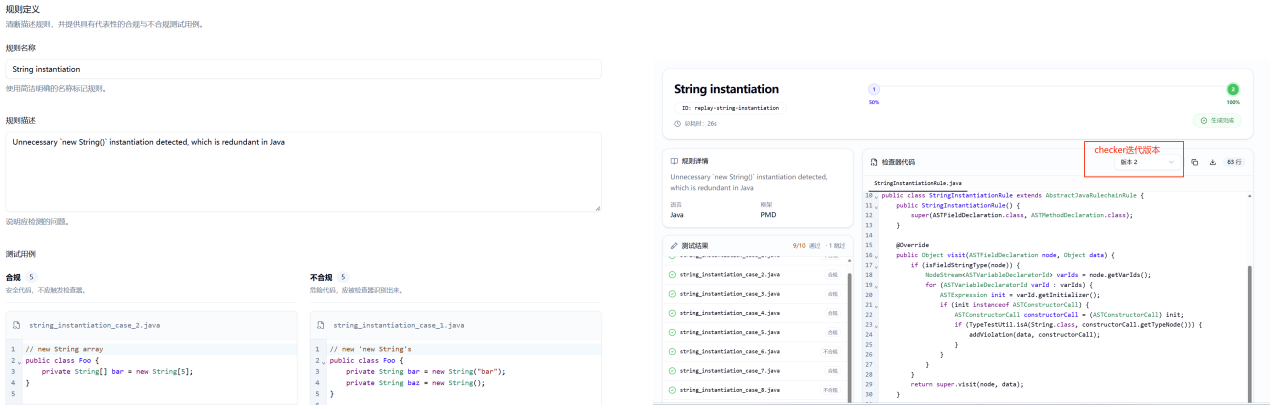
修改后的检查器需要重新编译，并在漏洞版本与修复版本上执行完整验证。若检测结果仍不符合预期，系统继续收集完整检测反馈及必要的局部探测结果，引导 Agent 进行下一轮修复，直至满足预设验证条件或达到迭代上限。通过结合完整检查器的检测结果与关键检测逻辑的局部匹配信息，DPP 为 Agent 提供了从最终告警到中间程序关系的诊断依据，使检查器修复能够针对具体检测条件进行调整。

4 用户界面

AutoChecker 同时提供命令行界面（CLI）与在线体验平台。CLI 面向开发人员，可在本地运行并集成到 CI/CD 流程；在线平台则将规则定义、测试用例确认、检查器生成和验证反馈组织为可视化工作流，降低非静态分析专业用户的使用门槛。用户既可以输入规则名称与自然语言描述，也可以提交规范文档或漏洞补丁，由系统提取检测需求并启动检查器生成任务。

如图 3 所示，在线平台围绕“测试驱动、反馈迭代”组织人机协同过程。在图 3a 中，用户定义待检测规则，并查看、补充或修改系统生成的合规与违规测试用例，以明确检查器应当接受和报告的代码边界。在图 3b 中，平台集中呈现规则信息、检查器源码及逐项测试结果；若生成版本未能通过全部用例，系统根据编译与测试反馈继续修复，并保留各轮迭代版本供用户切换查看。任务完成后，用户可以复制或下

载最终检查器代码，用于本地扫描或持续集成环境。



(a) 规则定义与测试用例确认　　(b) 生成结果、测试反馈与迭代版本

图 3: **AutoChecker** 在线体验平台的检查器生成流程

5　工具评估

5.1　实验案例与评价指标

本节从两个方面评估 **AutoChecker**：一是考察系统能否面向不同编程语言和静态分析引擎，将编码规范转化为符合测试预期的检查器；二是在一组 CVE 配对案例上，将漏洞与补丁驱动的检查器生成路线与已有方法进行比较，考察生成 CodeQL 安全检查器时的资源开销和迭代效率，实验所用大语言模型均为 DeepSeek-V4.1-Flash。

在规范与测试用例驱动实验中，我们构建了包含 180 条编码规则的规则集，覆盖 C/C++、Java、Python、JavaScript、TypeScript 和 Rust。针对 C++，系统从 GJB 8114–2013 规范文档中解析并结构化提取了 130 条规则；其余规则来自开源组织发布的编码指南。例如，部分 Python 规则取自 Open Source Security Foundation（OpenSSF）的 *Secure Coding Guide for Python*。我们为每条规则整理合规与违规测试用例，用于约束生成过程并检验检查器的实际判定结果。

漏洞与补丁驱动的检查器生成实验采用 9 个 Java CVE 构成案例集，覆盖 CWE-22、CWE-79、CWE-94、CWE-287 和 CWE-502。对比方法选用同样根据 CVE 信息和修复补丁生成 CodeQL 检查器的 QLCoder。9 个案例均具有 **AutoChecker** 与 QLCoder[1] 的完整运行记录，使我们能够在相同漏洞任务上逐项比较两个方法的 Token 消耗、运行时间、查询迭代次数和模型调用费用。

5.2　规范与测试用例驱动的检查器生成实验结果

表 1 汇总了 **AutoChecker** 在不同“分析引擎—编程语言”配置下的结果。对于每条规则，我们以判定结果符合预期的测试用例数除以该规则的测试用例总数，得到该规则的用例判定正确率；随后对同一配置下所有规则的正确率取算术平均。因此，表中的“平均用例判定正确率”是规则级宏平均，既要求检查器正确放行合规用例，也要求其正确报告不合规用例。耗时、Token 消耗和模型调用费用同样按规则取平均，其中 K 表示千 Token。

结果表明，**AutoChecker** 已在 Clang-Tidy、Clang Static Analyzer（CSA）、Semgrep 和 CodeQL 四种分析引擎以及六种编程语言上完成检查器生成与测试。在当前测试集上，Clang-Tidy 配置的平均用例判定正确率为 99.50%；Semgrep 在 Java、Python、Rust 和 JavaScript 上均为 100.00%，在 C/C++ 和 TypeScript 上分别为 95.00% 和 90.00%。相比之下，CSA 和 CodeQL-C/C++ 配置分别为 92.64% 和 80.00%，表明这些配置在当前规则集上仍有降低误报或漏报的空间。

不同配置的生成开销也存在明显差异。Semgrep-Java 的平均耗时、Token 消耗和模型调用费用最低，分别为 239.62 秒、34.90 K 和 0.13 元；CSA-C/C++ 的平均耗时和模型调用费用最高，分别为 3370.03 秒和 7.78 元；CodeQL-Python 的平均 Token 消耗最高，为 1434.89 K。由于各配置对应的规则集合、规

表 1: 规范与测试用例驱动的检查器生成结果

引擎	语言	平均耗时（s）	平均 Token（K）	平均模型调用费用（元）	平均用例判定正确率（%）
Clang-Tidy	C/C++	1363.15	159.75	0.64	99.50
CSA	C/C++	3370.03	970.68	7.78	92.64
Semgrep	C/C++	364.26	51.28	0.18	95.00
Semgrep	Java	239.62	34.90	0.13	100.00
Semgrep	Python	280.34	39.01	0.14	100.00
Semgrep	Rust	606.18	83.94	0.32	100.00
Semgrep	JavaScript	805.90	108.57	0.41	100.00
Semgrep	TypeScript	694.34	86.94	0.34	90.00
CodeQL	C/C++	746.03	414.05	0.31	80.00
CodeQL	Python	2777.81	1434.89	1.15	100.00

则复杂度和测试规模并不完全相同，这些数据仅用于描述当前实验中的资源开销，不能将差异完全归因于分析引擎或编程语言。

5.3 漏洞与补丁驱动的检查器生成效率

对于每个案例，**AutoChecker** 与 QLCoder 面对相同的 CVE 及修复信息，分别生成 CodeQL 检查器；实验记录从任务开始到获得最终检查器期间的资源消耗和迭代次数。

图 4 从总 Token 消耗、运行时间、查询迭代次数和模型调用费用四个维度比较 **AutoChecker** 与 QLCoder，四项指标均为数值越低越好。



(a) 总 Token 消耗

(b) 运行时间

(c) 查询迭代次数

(d) 模型调用费用

图 4: **AutoChecker** 与 QLCoder 在 9 个 CVE 上的生成效率比较

从 9 个 CVE 的配对均值看，**AutoChecker** 每个任务的平均总 Token 消耗为 804.70 K，QLCoder 为 11835.47 K，前者仅为后者的 6.80%；二者的平均运行时间分别为 20.02 分钟和 57.04 分钟，平均查询迭代次数分别为 2.22 次和 3.78 次，平均模型调用费用分别为 1.98 元和 12.22 元。换言之，在该配对子集上，**AutoChecker** 的 Token 消耗、运行时间、迭代次数和模型调用费用分别降低约 93.20%、64.90%、

41.18% 和 83.83%。逐项结果进一步表明，**AutoChecker** 在全部 9 个 CVE 上均使用了更少的 Token，并在其中 8 个 CVE 上取得更短的运行时间和更低的模型调用费用。

6 软件成果

为检验生成检查器在真实软件中的有效性，我们使用 **AutoChecker** 在 CWE-bench-Java 等数据集上生成检查器，并进一步扫描具有实际开发活动的开源代码仓库。根据目前已整理的 Issue、邮件报告与安全公告，**AutoChecker** 已在 **13 个开源仓库中发现并报告 16 项缺陷**，覆盖路径穿越、符号链接逃逸、命令注入以及敏感信息写入日志等多类安全问题, 大部分报告都获得了开发者确认，表 2 汇总了相关成果。

其中，两项由本项目发现并报告的问题已经获得 CVE 编号，表中以粗体突出显示：**CVE-2026-103754** 对应 **ansible/ansible-runner**，其归档接收函数未校验成员路径和符号链接目标，可能造成目标目录之外的文件写入或权限修改；**CVE-2026-71364** 对应 **ansible/awx**，其项目归档解压流程缺少规范化与目录边界检查，恶意归档可造成路径穿越和任意文件写入。两个漏洞均体现了 **AutoChecker** 将生成检查器用于真实开源项目后发现未知缺陷的能力，也说明自动生成规则不仅能够复现实验数据集中的已知模式，还能够迁移到实际工程代码中产生可验证的安全成果。

表 2: **AutoChecker** 在真实开源项目中发现并报告的缺陷

仓库名称	代码版本（前缀）	**CWE**
ansible/ansible-runner	**8dcdadb**	**CWE-22 / CWE-59**
seleniumbase/SeleniumBase	5300cae1	CWE-78
Flexget/Flexget	2c9593f	CWE-78
PaddlePaddle/PaddleDetection	b25522a	CWE-502
PaddlePaddle/PaddleDetection	b25522a	CWE-22 / CWE-59
megvii-research/NAFNet	2b4af71e	CWE-502
sczhou/CodeFormer	b33cc7d	CWE-502
XPixelGroup/BasicSR	8d56e3a	CWE-502
PaddlePaddle/PaddleSeg	3c4db66	CWE-22 / CWE-59
ansible/awx	**bbda905**	**CWE-22**
platformio/platformio-core	ff26956e	CWE-22
conda/conda	1c92362	CWE-532
conda/conda	1c92362	CWE-502
celery/celery	30402bd	CWE-502
scrapy/scrapy	7a62572	CWE-532
scrapy/scrapy	7a62572	CWE-502

7 相关信息

原创性. **AutoChecker** 项目中的核心组件, 包括 Agent 工作流、知识检索、反馈修复等机制, 均为自主研发。

发表情况. 本项目相关论文 [2] 已发表于国际会议 ICSE 2026, 论文中详细介绍了 **AutoChecker** 的创新点和核心贡献, 针对 PMD 工具进行了实验验证, 结果表明 **AutoChecker** 能够有效生成 PMD 检查器并发现真实缺陷。后续我们在此基础上进一步扩展了对 Clang-Tidy、CodeQL 和 Semgrep 等工具的支持, 并在真实开源项目中发现了多个安全缺陷。

开放访问. **AutoChecker** 项目的源代码、文档解析工具、知识检索模块和示例规则已在 GitHub 上开源, 地址为 `https://github.com/SQUARE-RG/AutoChecker`, 在线体验地址为 `https://autochecker.veilaxis.com`, 欢迎研究人员和开发者使用、反馈和贡献。

参考文献

[1] Claire Wang, Ziyang Li, Saikat Dutta, and Mayur Naik. QLCoder: A query synthesizer for static analysis of security vulnerabilities. In *The Fourteenth International Conference on Learning Representations*, 2026.

[2] Jun Liu, Yuanyuan Xie, Jiwei Yan, Jinhao Huang, Jun Yan, and Jian Zhang. Write your own codechecker: An automated test-driven checker development approach with llms. In *Proceedings of the International Conference on Software Engineering (ICSE)*, 2026.